看门狗

硬件超时复位与业务健康监控

嵌入式系统编程与实践

刘君榕

2026/10/28



业务停滞与系统恢复

北京大学 PEKING UNIVERSITY

无人值守的采集设备

1. 正常：采集数据，处理，更新结果。
2. 故障：一直等待新数据，结果不再更新。
3. 需求：异常后自动重启，减少人工干预。

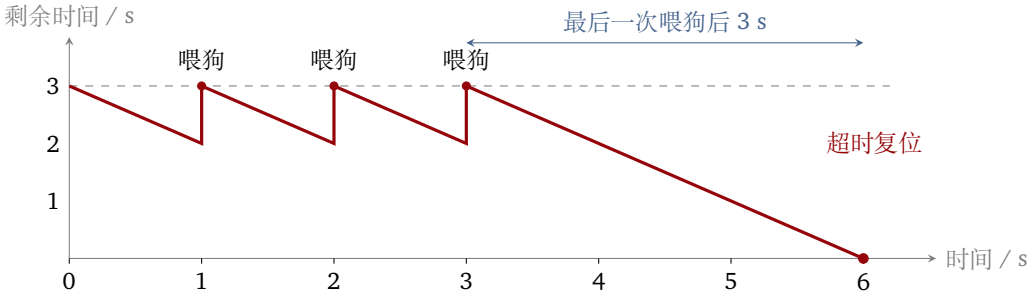
看门狗的判断依据

软件定期**喂狗**，刷新硬件计数器。

超过期限没有刷新，硬件发出**复位请求**。

业务是否正常，取决于软件在什么条件下允许喂狗。

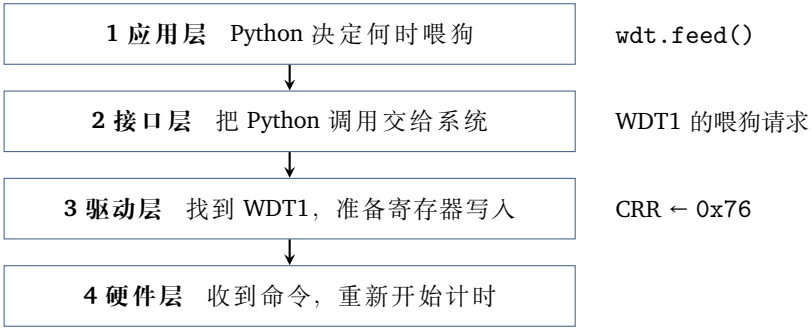
喂狗与超时的时间关系



`timeout = 3 s` 第 1、2、3 秒喂狗，之后停止。

每次喂狗都重新开始计时，最后一次喂狗后 3 秒到期。

一次喂狗经过的四个层次



软件把请求一层层交下去，硬件收到命令后继续计时。

1 应用层：决定何时喂狗

参数形式 `WDT(id, timeout)`

```
from machine import WDT

wdt_id = 1
timeout_s = 3
wdt = WDT(wdt_id, timeout_s)

wdt.feed()
```

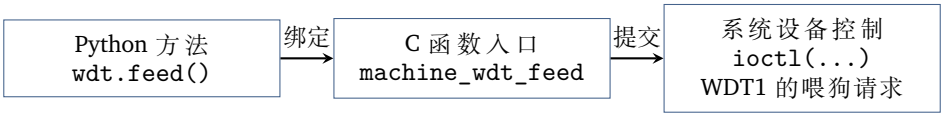
`id`：看门狗模块编号。
K230 有 WDT0、WDT1 两路。

本例为何取 1？
这套 CanMV 用户接口使用 WDT1。

`timeout`：超时时间，单位秒。
本例 `timeout_s = 3`。

收到：程序决定本次需要喂狗。
交给下一层： `wdt.feed()`（请求喂狗）。

2 接口层：把 Python 调用交给系统



绑定：把 Python 的 `feed` 方法对应到 C 函数。

提交：目标设备是 WDT1，操作是喂狗。

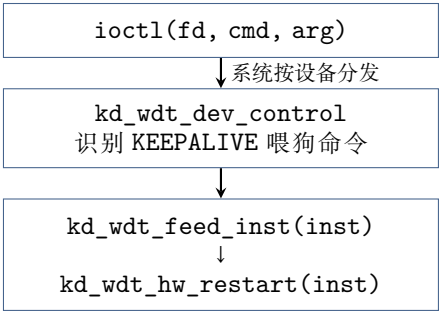
`ioctl(...)` 是设备控制入口；它把请求送入操作系统。

收到：`wdt.feed()`（请求喂狗）。

交给下一层： WDT1 的喂狗请求。

3 驱动层：把喂狗请求变成寄存器写入

请求怎样进入驱动



下一步：通用寄存器写入形式

```
kd_wdt_writel(inst, reg, value);
```

`inst`：操作哪个 WDT 实例。
`reg`：寄存器偏移（字节）。
`value`：写入的 32 位数据。

代入本次喂狗操作

`reg = 0x0C`：选 CRR 重启寄存器。
`value = 0x76`：重启计数命令。

```
kd_wdt_writel(inst, 0x0C, 0x76);
```

收到：WDT1 的喂狗请求。

交给下一层： CRR ← 0x76（写计数器重启寄存器）。

4 硬件层：写入命令后重新开始计时

1 应用层 → 2 接口层 → 3 驱动层 → **4 硬件层**

收到：CRR ← 0x76。CRR 是计数器重启寄存器。

WDT_CRR

Offset=0x0c

Bits	Name	Reset	Access	Descriptions
7:0	WDT_CRR	0	W	Counter Restart Register. This register is used to restart the WDT counter. As a safety feature to prevent accidental restarts, the value 0x76 must be written. A restart also clears the WDT interrupt. Reading this register returns zero. **Values:** - 0x76 (RESTART): Watchdog timer restart command Exists: Always

硬件手册：地址分布（表 1-2）；WDT 计数与寄存器（§2.7）

物理地址 0x9110680C ← 32 位值 0x00000076

计时由 WDT1 内部的专用 32 位倒计数器完成。

分频原理与超时时间

分频降低计数时钟频率，同样的计数周期数需要更长时间。

$$f_{\text{cnt}} = \frac{f_{\text{root}}}{d} \qquad T = \frac{N}{f_{\text{cnt}}} = \frac{N\,d}{f_{\text{root}}}$$

四个关键变量

f_{root}（`root_hz`）	分频前根时钟，本例 24 MHz
d（`div`）	分频数，范围 1–64
k（TOP / `idx`）	计数档位编号，范围 0–15
N（`cycles`）	$N = 2^{16+k}$ 个计数周期

请求 3 s 的配置示例

选 TOP $k = 5$，即 $N = 2^{21}$。
选 $d = 35$，计数时钟约 685.7 kHz。

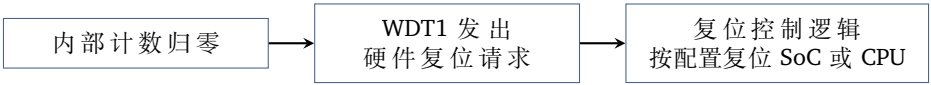
代入公式，$T \approx 3.058$ s。
34 分频约 2.971 s，不足 3 s。
分频字段写 34，表示 35 分频。

驱动遍历分频与 TOP，选取不短于请求且最接近的组合。
100 MHz 为总线访问时钟。

倒计时寄存器与超时复位

WDT1 寄存器物理地址与直接复位流程。

寄存器	物理地址	作用与配置示例
WDT_CR	0x91106800	bit0=1：启用；bit1=0：超时直接请求复位
WDT_TORR	0x91106804	写入 5：选择 2^{21} 个时钟的超时档位
WDT_CCVR	0x91106808	**只读**当前计数：$\cdots \to 3 \to 2 \to 1 \to 0$
WDT_CRR	0x9110680C	写 0x76：重新装载、继续计数；读回为 0



0x91102060[4:3] 预先选择复位范围：0/1 → SoC；2 → CPU0；3 → CPU1。

喂狗是重新计时；计数归零后由硬件连线请求复位，无需 CPU 再执行代码。

K230 与 K210：WDT 接口完整对比

对比项	K230	K210
构造及默认值	`WDT(id=1, timeout=5, *, auto_close=True)`	`WDT(id=0, timeout=5000, callback=None, context=None)`
编号 id	用户通路使用 WDT1	0、1（两个 WDT）
单位 / 3 s	整数**秒**，`WDT(1, 3)`	整数**毫秒**，`WDT(0, 3000)`
启动与喂狗	构造时启动，`feed()` 刷新计时	构造时启动，`feed()` 刷新计时
回调与上下文	未提供 `callback/context`	`callback(wdt)` 接收 WDT 对象 `wdt.context()` 取得上下文
释放或停止	`close()`，`stop()` 为同一入口 释放用户控制，内核接管喂狗	`stop()` 停用对应 WDT 并关闭对应时钟

现场演示

北京大学 PEKING UNIVERSITY

VS Code 实机演示

采样器的超时选取示例

采样器每完成一轮采样和处理后喂狗，T 为超时时间（单位：秒）。

正常时，不能误复位
采样最多 0.8，处理 0.3，调度延迟 0.1。

$$G = 0.8 + 0.3 + 0.1 = 1.2$$

G 是正常时两次喂狗的最长间隔。
本例另留 50% 余量：$M = 0.6$。

所以要求：$T > G + M = 1.8$。

故障后，5 秒内恢复采样
业务要求决定允许停机时间：$D = 5$。
判断故障最多 0.5，复位到恢复采样最多 1。

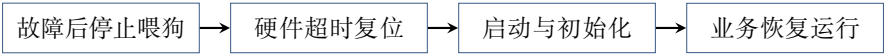
$$R = 0.5 + 1 = 1.5$$

等待看门狗到期的时间另计为 T。

要求 $T + R \le D$，即 $T \le D - R = 3.5$。

$$G + M < T \le D - R \quad \Rightarrow \quad 1.8 < T \le 3.5$$

复位后的业务恢复



- 在开机入口自动启动业务，完成外设和状态初始化。
- 保存复位原因和故障记录，定位异常来源。
- 设置重试上限，连续失败时进入安全状态。

开机自启动完成恢复，重试上限控制连续重启。

看门狗的功能与意义

看门狗在喂狗超时时**触发复位**，提供异常后的自动重启机制，提高系统可用性和无人值守运行能力。

在自己的项目里使用看门狗

K230：按业务进度喂狗

```
from machine import WDT
import time
init_peripherals()
wdt = WDT(1, 3)
seq = last_seq = 0
while True:
    try:
        if sample_and_process():
            seq += 1
    except Exception:
        pass
    if seq != last_seq:
        wdt.feed()
        last_seq = seq
    time.sleep_ms(10)
```

接上自己的外设和业务

`init_peripherals()`：外设初始化。

`sample_and_process()`：一轮采样处理成功，才返回 `True`。

进度变化才喂狗

失败或异常不更新 `seq`；没有新进度，就不调用 `feed()`。

超时按自己的业务选

代码以 3 秒为例。按正常最坏间隔、余量和恢复要求确定超时。

停止刷新 → 超时复位 → 开机恢复。